

《挖掘机拯救世界》

从 Wanderburg 参考到 Steam 独立游戏计划

Godot 4.6.3 · Windows Steam 单机 · 个人主导 + AI 协作 + 必要外包

研究与立项基线 · 2026-09-20

《挖掘机拯救世界》：从 Wanderburg 参考到 Steam 独立游戏计划

研究时间：2026-09-20 | 研究对象类型：本地游戏发布包 + 原创游戏立项 | 技术路线：Godot 4.6.3 / Windows Steam 单机

给你的结论

可以把 Wanderburg 的“移动载具、目标选择、局内构筑、局外解锁、明显体量成长”作为设计参照，但不能把 `F:\SteamLibrary\steamapps\common\Wanderburg Game` 当作可打开的工程。这个目录是 Unity 6000.0.63f1 IL2CPP Windows 发布包，没有 `project.godot`、`ProjectVersion.txt`、C# 源文件或 Unity 编辑器工程标记；它能帮助我们识别已部署的系统边界和体验规律，不能还原原始场景、Prefab、算法或资产授权。

你的游戏建议定为：3D 斜俯视载具动作 Roguelite，玩家驾驶挖掘机清除污染机械、回收废料、安装临时模块，完成修复目标后让河流、风机、道路和城市重新运转。“挖掘”必须改变路线和战斗，不是一个只换皮的射击游戏；“修复”必须能在当局和世界地图上看见，而不是结算时的一张图片。

个人主导、AI 协作、必要美术/音频外包的现实基线是 12 – 18 个月。先做 2 – 3 周灰盒手感闸门，再决定是否量产。首发上限建议 3 个群系、9 个合同、1 个终局、3 个共享结构底盘、6 个作业副工具、8 个辅助模块、8 个普通敌人、3 个区域首领和一套组合终局；如果产能不够，减为 2 群系、6 合同、2 底盘、4 工具、6 模块、2 首领，保留完整结尾、手柄、存档和性能。

一、参考工程究竟告诉了我们什么

能验证的本地证据

已运行工作区脚本 `tools/audit_reference_release.py`，只读检查发布目录并输出 `docs/reference-audit.json`：

项目	证据	对本作的意义
引擎	<code>Wanderburg_Data\data.unity3d</code> 头部含 <code>6000.0.63f1</code>	参考作品走 Unity 6，不意味着本作要改用 Unity
发布形态	<code>UnityPlayer.dll</code> 、 <code>GameAssembly.dll</code> 、 <code>il2cpp_data/Metadata/global-metadata.a.dat</code>	IL2CPP 只留下部署产物，不能拿来当源工程
资源加载	<code>StreamingAssets/aa/catalog.bin</code> 、 <code>Addressables 2.7.6</code> 设置、 <code>AddressablesLink/link.xml</code>	内容按可寻址资源组织；本作可用 Godot <code>PackedScene/Resource</code> ，但也要有稳定 ID
输入/画面/内容服务	打包程序集出现 <code>Input System</code> 、 <code>Cinemachine</code> 、 <code>URP</code> 、 <code>Localization</code> 、 <code>FMOD</code> 、 <code>Steamworks/Heathen</code>	可推断这些是部署依赖；不能证明每个类都参与核心玩法
应用类型名	元数据表中出现 <code>OverworldManager</code> 、 <code>MapGenerator</code> 、 <code>WaveSpawner</code> 、 <code>AttackModuleV2</code> 、 <code>ModuleUpgrade</code> 、 <code>ArtifactSystem</code> 、 <code>QuestSystem</code> 、 <code>SaveGame</code> 、 <code>SaveLoad</code> 、 <code>AchievementManager</code> 等	可以用来画“移动载具 + 波次 + 模块 + 任务 + 局外进度”的系统假设图
部署规模	87 个文件，约 703 MiB；发布包不是可编辑项目	只能做行为与系统研究，不复制它的二进制或素材

元数据还保留了很多 `Assets/Scripts` 路径名，例如模块、载具、地图、任务、统计、存档、UI 和 Steam 适配的分类。这些名字是构建中嵌入的字符串，不是源文件内容，也不能证明类之间的调用关系。报告把它们作为“命名证据”，没有把它们伪装成已经读懂的实现。

玩法层的可靠参照

Wanderburg 官方 Steam 页面明确写出：驾驶移动城堡，吞噬较小目标使城堡扩张，装载模块，局外解锁模块、载具和船长，并支持键鼠和手柄。这个闭环最值得借鉴：移动本身是战斗，环境目标是成长资源，装配会改变下一次决策。

官方更新记录还暴露了独立游戏最容易晚修的部分：手柄驾驶方式、敌人卡住、首领出生位置、越界保护、性能、升级软锁、结算成就、分数合法性和存档/恢复的边缘情况。我们的 M0/M1 就要把这些问题列为验收线，而不是等做完三张地图再处理。

参考边界：不复制 Wanderburg 的城堡、名称、角色、素材、音频、代码或数值；不把任何社区 Wiki 的内部数值当作源码事实。其 Steam 页面只用于公开设计定位与产品层信息。

二、原创游戏的完整体验

核心循环

玩家在一个 12 – 18 分钟的“工程合同”中循环：驾驶寻找目标 → 主挖斗铲击/挖起 → 携带块料或回收废料 → 投掷、冲刺或拆除更硬障碍 → 三选一升级 → 修复一个设施 → 进入更危险的区域 → 击败首领 → 看到世界恢复并带回蓝图。

在 5 – 15 秒的微循环里，玩家应同时得到接触特效、机械音、目标状态变化和资源回收；在 12 – 18 分钟的局内循环里，路线、挖掘风险和升级选择应互相影响；在整个战役里，修复的河岸、风机和城市照明会持续改变世界地图。

第一场 15 分钟

0:00 – 0:30 玩家从被掩埋的车库开出，电台只要求先铲开出路；0:30 – 1:15

清理三组松散废料和一只维修爬虫；1:15 – 2:00 给出宽斗、快速液压、废料投掷三选一；2:00 – 3:30

铲开水泵入口并看到第一块土地变绿；3:30 – 5:30 开启第二体量阶段和副工具槽，出现冲锋敌人；5:30 – 7:00

清理垃圾坝，支路提供蓝图；7:00 – 9:00 用已有动作击败压路机首领；9:00 – 10:00 河岸复绿、鸟声恢复；10:00 – 12:00

结算解释贡献、死因和新蓝图；12:00 – 15:00 自愿开第二合同。

新玩家应该在 3 分钟内完成一次挖掘、投掷或升级，并能说出“我挖出来的东西为什么有用”。如果世界修复没有提供安全区、捷径或补给，只改变颜色，就要回到原型重做。

载具与成长

三种底盘共享运动代码、挖臂骨架和槽位：均衡履带、轻型宽轮、重装履带。成长只做三个经过验证的阶段，固定主挖斗永远存在；一槽作业副工具、两槽辅助模块逐步打开。六种副工具分别是液压破碎锤、电磁回收盘、螺旋钻、高压水炮、碎料发射器和电弧焊接器。八种模块负责料斗容量、液压节奏、散热、装甲、维修、雷达、冲刺与工程目标联动。

三条必须可通关的构筑：近身冲铲、聚怪投掷、边挖边射。升级不是只加伤害：它要改变最佳路线、铲击角度、回收范围、危险处理或修复效率。每次三选一至少有一项与当前工具或挖斗相关，不能出现没有槽位或没有即时收益的选项。

模块组合是本作的第一设计支柱：模块分为 Tool、Drive、Reaction、Repair、Morph

五类，通过直接组合、条件组合和转化组合改变输入语义、数值转换、Prefab 轮廓和修复反馈。AssetRipper

导出的参考 Prefab 显示，模块确实把主动/被动实例化点、冷却层、T0 – T5

视觉层级、安装闪光和槽位资格放在同一个装配对象里；本作会把这些责任拆成

ModuleDef、ModuleVisual、ModuleRuntime 和 LoadoutAssembler，避免“只加

DPS”的升级。首个原型只验证宽斗+磁吸、水炮+电弧、惯性飞轮+冲刺三条组合。

世界与叙事

旧世界的自动建设系统把“持续施工”当作最高指令，废料填满河道、工厂吞掉农田。玩家驾驶最后一台能自行选择任务的工程车，帮助维修师、生态观察员和逐渐改变的调度 AI

恢复水、电与通路。每合同只需要出发一句、过程最多两次短讯、胜利一段，不做高成本全语音过场。

三群系为河谷、盐海、山城，每个三合同；终局复用三地模块解决总控工地。任务语法收敛为清障开路、解除/修复节点、击败目标、短时守住启动设施四种，避免护送脆弱 NPC 和长距离交货把节奏拖成模拟经营。

三、技术栈与系统结构

已锁定的技术决策

层	决策
引擎	Godot 4.6.3 stable；项目文件和 export templates 同补丁版本
代码	静态类型 GDScript；只有 profiler 证明必要时才将纯计算模块移入 GDExtension
画面	3D 斜俯视、Mobile 渲染器制作基线、低多边形共享材质与实例化植被
物理	Jolt、固定 60 Hz；CharacterBody3D 街机底盘，碎片才用短寿命刚体
挖掘	固定地面 + 1 m 逻辑格的分块覆盖层/预制破坏物；不做任意体素、流体或多关节刚体
数据	自定义 Resource .tres、PackedScene .tscn、稳定字符串 ID
音频	AudioStreamPlayer/3D + AudioServer 总线；首版不引入 FMOD workflow
资产	Blender 5.2.1 LTS → GLB；统一米制、枢轴和 socket_arm/tool/support_l/support_r
平台	Windows x86_64 首发；Steam Deck/Proton 先实机测试再宣传
Steam	GodotSteam GDExtension，隔离 SteamAdapter 与离线适配器；插件版本在 M1 兼容试验后锁定

运行时结构

AppRoot 管启动、菜单、营地、合同和结算；Services 管存档、设置、内容目录、平台和音频；Session 只拥有当前 run，内含

RunController、World、EncounterDirector、EntityRegistry、PlayerVehicle 和 RunState；UI 只通过命令和稳定 ID 与系统交互。不要把敌人、掉落或局内升级做成全局单例。

挖掘动作按“输入 → 射程/硬度/资源校验 → attack_id + target_id 去重 → 更新格子/物体和账 → 标记 chunk dirty → 分帧更新视觉、碰撞、导航 →

播放表现”的顺序执行。碎片、尘土和音效永远是表现层，不能成为权威库存。

VehicleDef、ToolDef、ModuleDef、UpgradeDef、EnemyDef、MissionDef、RepairDef 和 DropTable 只保存定义；当局状态独立存放，避免共享 Resource 被写入血量或冷却。程序生成对象 ID 使用 run_id + chunk + spawn_slot，不要把 NodePath、实例号或 RID 写入存档。

存档与结算

存档使用 user:// 的一个版本化 envelope，内含玩家档案与当前 run；A/B 双槽、序号、父提交、提交 ID、校验值和迁移版本保证崩溃可恢复。局内只在安全检查点保存地图变更集、库存、升级和任务状态，不承诺逐帧恢复飞行碎片。

结算状态严格走 ACTIVE → RESULT_PENDING → RESULT_COMMITTED：先冻结合同并生成唯一 result_tx_id，把奖励、蓝图、修复状态、已处理事务和 run

标记一次写入；确认磁盘提交后才播放到账动画。崩溃重启重复看到 pending 时重试同一事务，已提交则直接显示结果，不重复发钱。成就和 Steam 调用在本地提交后异步重试，平台不可用不能阻塞离线进度。

四、开发路径与验收

2–3 周原型闸门

第 1 周只做一个灰盒底盘、转台、两段挖臂、主斗、路障、土堆和三种输入；第 2 周加入三个改变行为的升级、一个可被清障打断的敌人和一个修复节点；第 3 周让 5–8 位未参与开发者独立试玩 15–20 分钟。通过线是：多数人 30 秒内理解铲击、3 分钟内完成升级；至少三人主动再来一局或换构筑；键鼠/手柄都能完成；体量变化不导致卡路；问题能归因于可修复的反馈或控制。

不通过时只允许带假设的 1–2 周返工：减少按键、加强主斗、自动辅助搬运、缩小可变地形。仍不通过就改核心方向或暂停商业化，不能先买几十套美术。

12–18 个月交付顺序

M0 手感原型；M1 4–8 周完成一个可恢复的小合同、Windows 导出、手柄、结算事务和 Steam 兼容试验；M2 第 3–4 月做达到最终品质的河谷垂直切片；M3 第 5–8 月量产前两群系和工具/敌人；M4 第 9–11

月完成第三群系、终局、开场和结局并冻结系统；M5 第 12–14

月做数值、优化、无障碍、本地化、云存档和成就；M6 第 15–18 月做发布候选、干净安装、旧档迁移和 Steam 发包缓冲。

Steam 接入从 M1 开始：导出包离线也能启动，实际客户端验证 Overlay、Steam Input、云存档和成就；商店素材在 M2 有真实垂直切片后制作，发布前重新核对 Steam Direct 规则，不把不断变动的政策写死在游戏逻辑里。

性能与发布目标

设计目标是 Windows 1080p 中画质持续 60 fps，战斗常规 30–50、压力 80 个交互敌人，64 个短寿命动态碎片，连续 20 次合同加载/卸载无内存持续增长；Steam Deck 的 1280×800/40 fps 只在真机验证后决定。四个固定 benchmark 为作业场、战斗场、破坏峰值和存档生命周期，验收一律以 release 导出包为主，不能用本机 4070 Ti SUPER 的表现当最低配置承诺。

五、美术、音频与资产边界

视觉方向是低多边形工程玩具：工程黄挖掘机、深蓝灰车架、危险珊瑚红、回收青、修复草绿。辨识顺序固定为主斗/驾驶室→副工具→敌方危险区→可挖资源→

修复设施。污染、清理、修复用三态地块和少量水、鸟、植被实例表达，避免全地图低成本模拟。

首发资产上限是 3 底盘×3 阶段、1 套挖臂骨架、6 副工具、8 模块、8 普通敌人、3 首领、54 个环境结构模块、24 个通用废土道具、9 套主要任务设施、约 72 个内容图标、约 20

个玩法特效模板。正式模型必须通过真实镜头、碰撞、枢轴、材质、暂停/受击/修复状态和授权台账六项验收。

动画优先机械节点与程序旋转：待机液压、起步刹车、瞄准、铲击接触、装料、投掷、受击、过热、体量扩张和报废；音频优先履带、电机、液压、金属/土石接触、危险预告、修复成功与五至六个音乐层。美术与音频预算初估 114–178 人日，另预留约 20% 集成返工；外包先买“一辆车+一套小景”的样件，确认风格和插槽后再量产。

六、现在就做的 10 件事

1. 在新 Godot 项目中锁定 4.6.3 和导出模板，建立 Git/LFS、game/、source_assets/、tools/、docs/ 目录。
2. 做 M0 灰盒：一个底盘、主斗、三个可破坏物、三个敌人、两个升级、一个水泵修复。
3. 同一场景同时按键鼠和手柄，不等以后再补。
4. 从第一天记录作业、战斗、破坏峰值 benchmark，先测 release 导出。
5. 建立 Resource 定义、稳定 ID 和数据校验器，禁止展示名称作为存档主键。
6. 先实现 A/B envelope、result_tx_id 和故障注入，再做第二合同。
7. 做一套可见的污染→修复三态地块和世界地图，不先画剧情插画。
8. 用真实镜头评审一阶段 II 挖掘机、一个工具、两种敌人和河谷小景。
9. 召集 5–8 名外部试玩者，作者不口头引导，记录具体卡点而不是只收总体好评。
10. 只有 M0/M1/M2 闸门通过后，才签批量美术、首领和宣传素材。

相关文档与证据

- [游戏设计基线](#)：完整核心循环、首 15 分钟、升级、合同、经济、叙事和减配线。

- [技术架构](#)：Godot 节点边界、分块挖掘、存档事务、性能预算、Steam 导出和风险。
- [开发路线](#)：个人制作的阶段、门槛、AI/本人/外包分工和周节奏。
- [美术与音频生产](#)：资产数量、风格、命名、GLB 接口、LOD、音频和外包台账。
- [模块组合与幻想载具设计](#)：模块类别、组合拓扑、操控变化、数值结构、Prefab 装配和 M0 组合原型。
- [AssetRipper 深度分析](#)：5 个场景、749 个 Prefab、模块层级、载具槽位、升级预览、操控字段和存档结构的导出证据。
- [参考包审计 JSON](#)：对发布目录的只读文件、程序集、IL2CPP 元数据和嵌入路径名审计。
- [Wanderburg Steam 官方页](#)；[Dome Keeper](#)；[Wall World](#)；[Terra Nil](#)；[Driveloop](#)。

这份计划的第一项真正产物不是一张美术图，而是 M0 灰盒中让挖斗同时成为武器、工具和修复钥匙。只要这一点成立，后面的群系、升级和宣传素材才有可靠的共同核心。